



Space, Time, Extremes and Geometric Deep Learning

Tanujit Chakraborty

Associate Professor of Statistics & Data Science

Sorbonne University



**“Space-time-causation, or name-and-form,
is what is called Maya.”**

— Swami Vivekananda

Time and Space

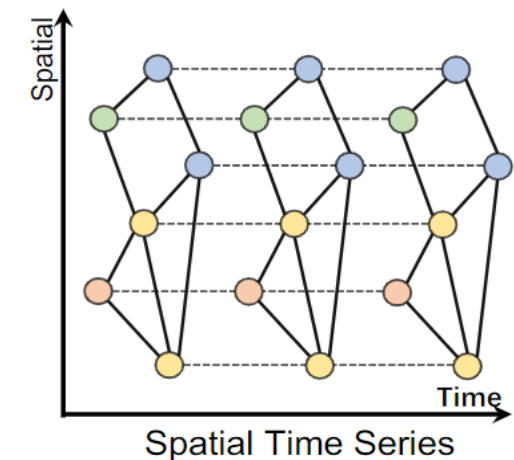
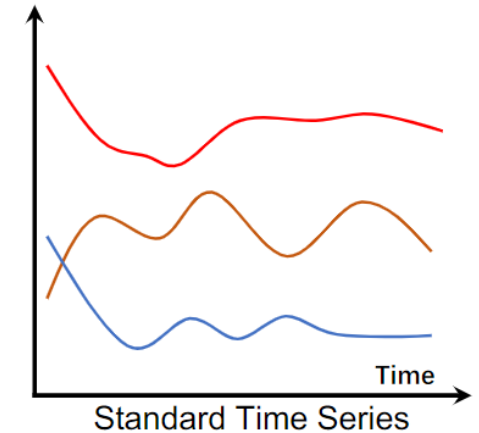


Time series is a set of observations, each one being recorded at a specific time.

Forecasting is estimating how the sequence of observations will continue into the future.

Spatio-temporal data is a type of data that combines spatial and temporal dimensions, capturing how measurements or observations vary across different locations over time.

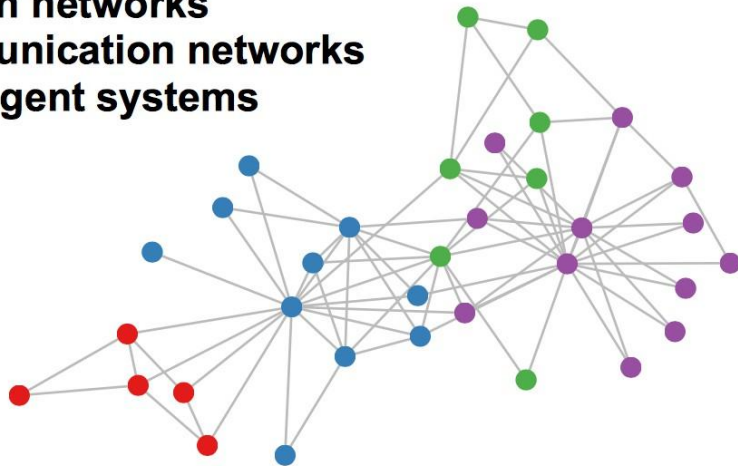
Spatiotemporal data can be modeled as **graph-structured data**, particularly when the data involves complex relationships, such as sensor networks, traffic flow, or social interactions.



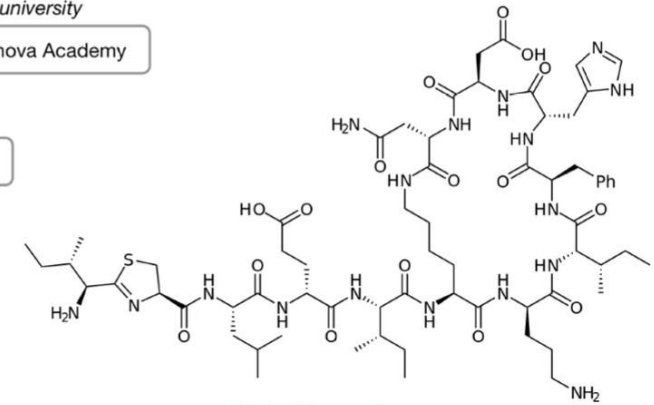
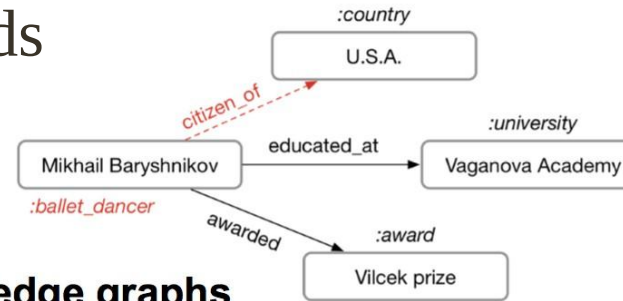
Graph-structured Data

A lot of real-world data does not “live” on grids

Social networks
Citation networks
Communication networks
Multi-agent systems

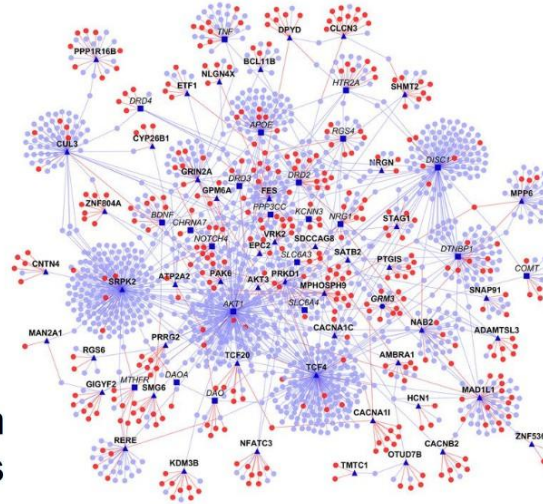


Knowledge graphs



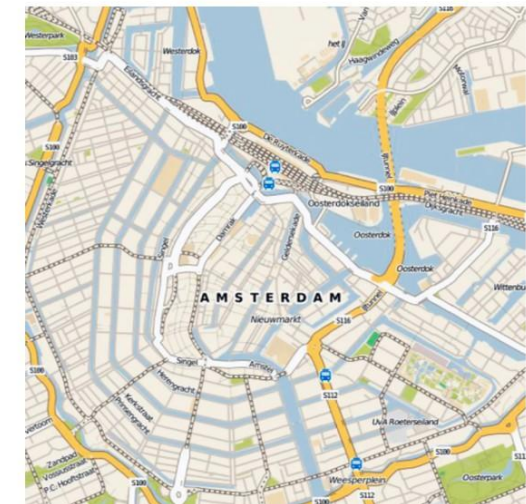
Molecules

Protein interaction networks



Standard CNN and RNN architectures don't work on this data

Road maps



What is a Graph?

- **Graph** represents the relations (edges) between a collection of entities (nodes) quantified using weights.
- Graph $G(V, E, W)$: V = nodes, E = edges (link node pairs), W = weights associated with edges.
- **Adjacency matrix (A)** of a graph is a sparse matrix with non-zero elements denoting graph weights.

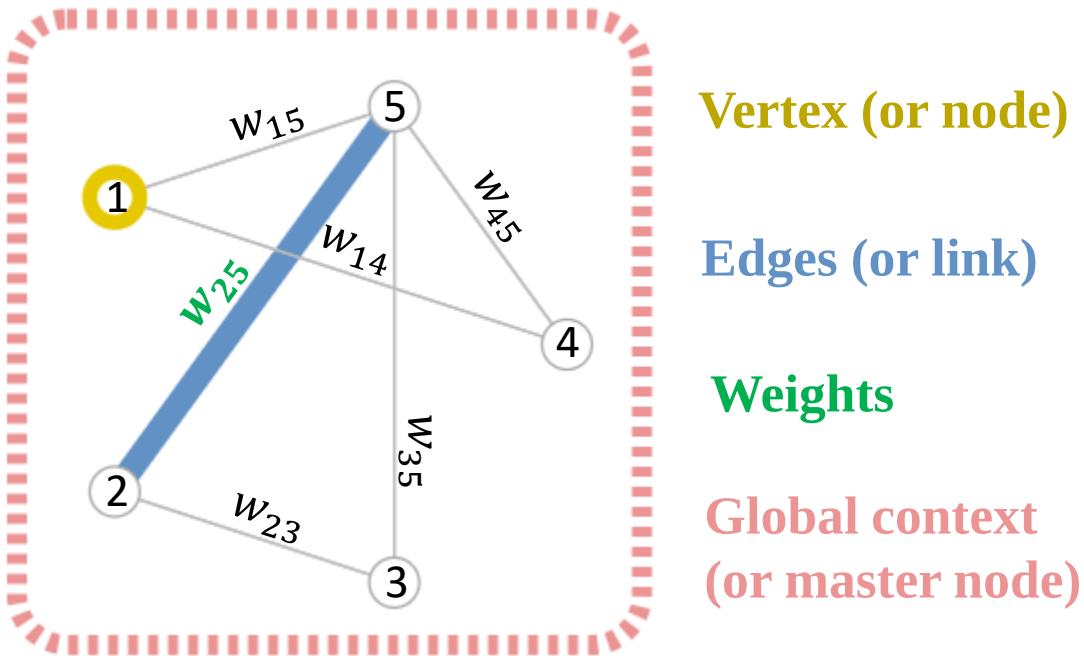


Fig: Attributes of undirected weighted graph

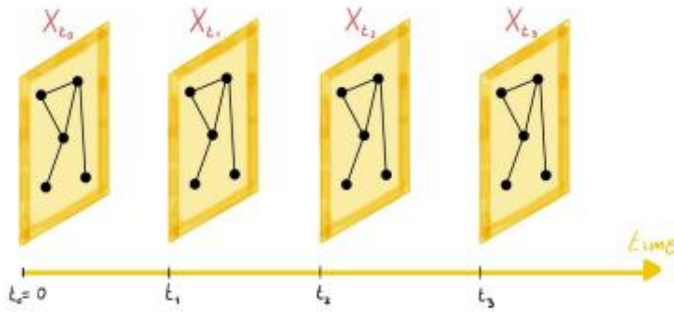
$$A = \begin{bmatrix} 0 & 0 & 0 & w_{14} & w_{15} \\ 0 & 0 & w_{23} & 0 & w_{25} \\ 0 & w_{23} & 0 & 0 & w_{35} \\ w_{14} & 0 & 0 & 0 & w_{45} \\ w_{15} & w_{25} & w_{35} & w_{45} & 0 \end{bmatrix}$$

Adjacency matrix of a symmetric **weighted static graph**

Dynamic Graphs

A dynamic graph is a type of graph whose structure (nodes and edges) and/or associated properties (features of nodes, X_t) change over time.

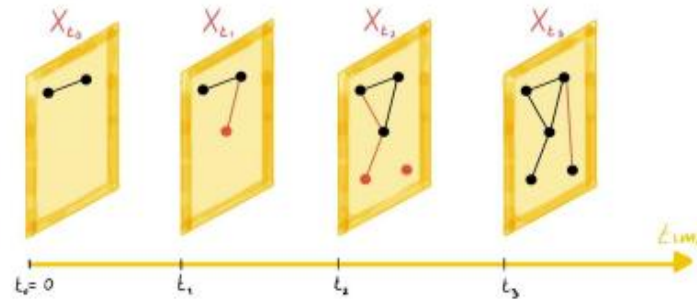
$$G_t = (V, E, X_t)$$



Spatio-Temporal Graph

- Topology is fixed, but features change over time
- (Usually) observed at regular intervals
- Examples: Air pollution level

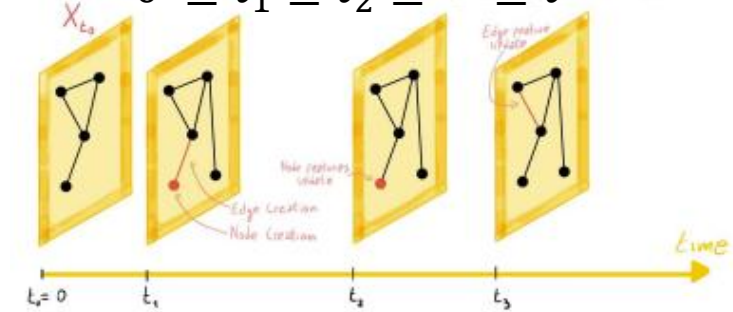
$$G_t = (V_t, E_t, X_t)$$



Discrete-Time Dynamic Graph

- Both topology and features change over time
- However, graph is observed at regular intervals
- Examples: Collaboration network

$$G(t) = \{x_{t_1}, x_{t_2}, \dots\}$$
$$0 \leq t_1 \leq t_2 \leq \dots \leq t$$

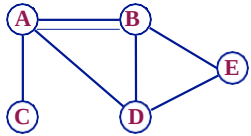


Continuous-Time Dynamic Graph

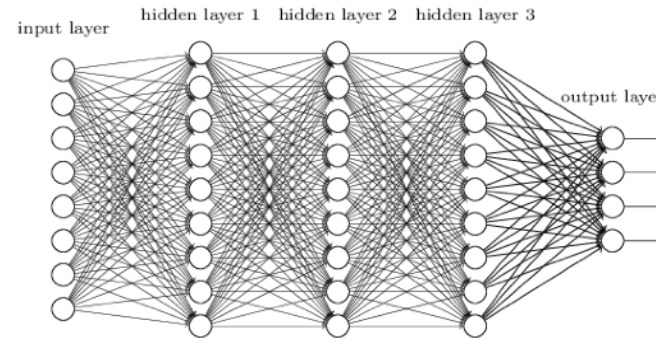
- Most general formulation
- Each change in the graph is observed individually with its timestamp
- Examples: Protein-protein interaction network

A naïve approach

- Take adjacency matrix $\mathbf{A}$ and feature matrix $\mathbf{X}$
- Concatenate them $\mathbf{X}_{\text{in}} = [\mathbf{A}, \mathbf{X}]$
- Feed them into deep (fully connected) neural net



	A	B	C	D	E	Feat	
A	0	1	1	1	0	1	0
B	1	0	0	1	1	0	0
C	1	0	0	1	0	0	1
D	1	1	1	0	1	1	1
E	0	1	0	1	0	1	0



? We need weight sharing!

Problems:

- Huge number of parameters
- Re-train if graph changes

Possible Solution:
Deep Learning on graphs or
Graph Neural (Convolutional) Networks

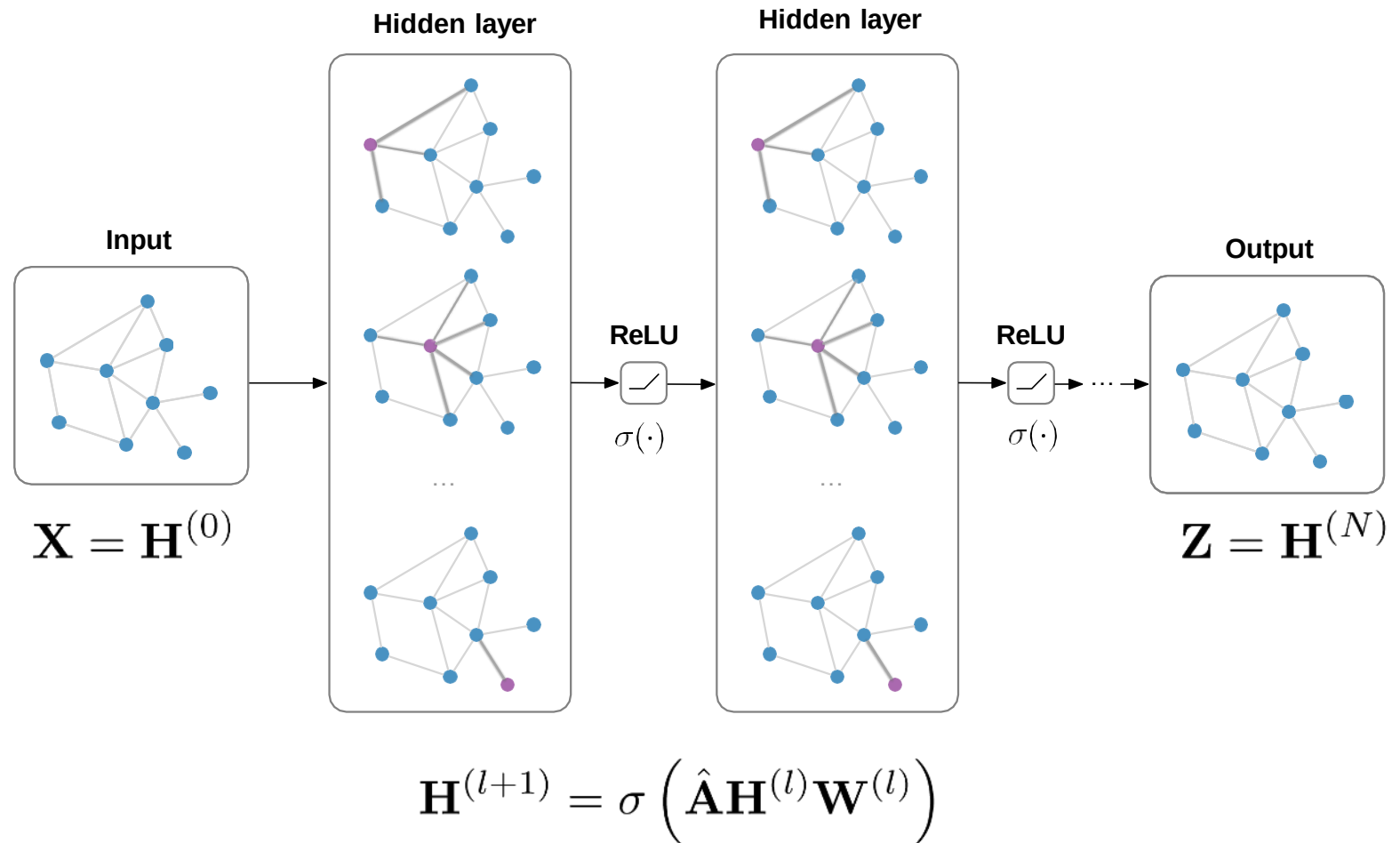
Graph Neural Networks (GNNs)

Notation: $\mathcal{G} = (\mathbf{A}, \mathbf{X})$

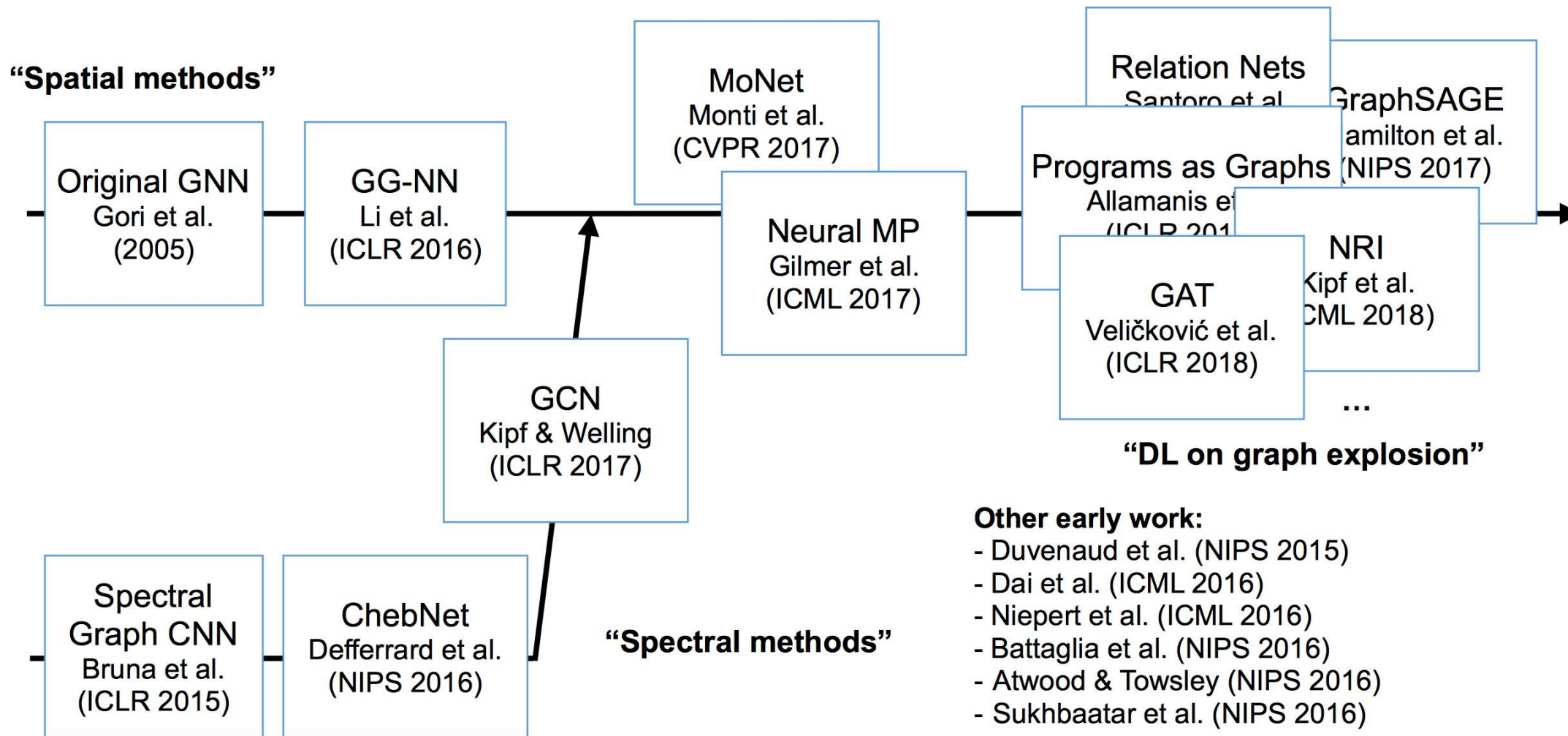
- Adjacency matrix $\mathbf{A} \in \mathbb{R}^{N \times N}$
- Feature matrix $\mathbf{X} \in \mathbb{R}^{N \times E}$

Main Idea: Pass messages between pairs of nodes and agglomerate

Alternative Interpretation: Pass messages between nodes to refine node (and possibly edge) representations



A Brief History of Graph Neural Nets



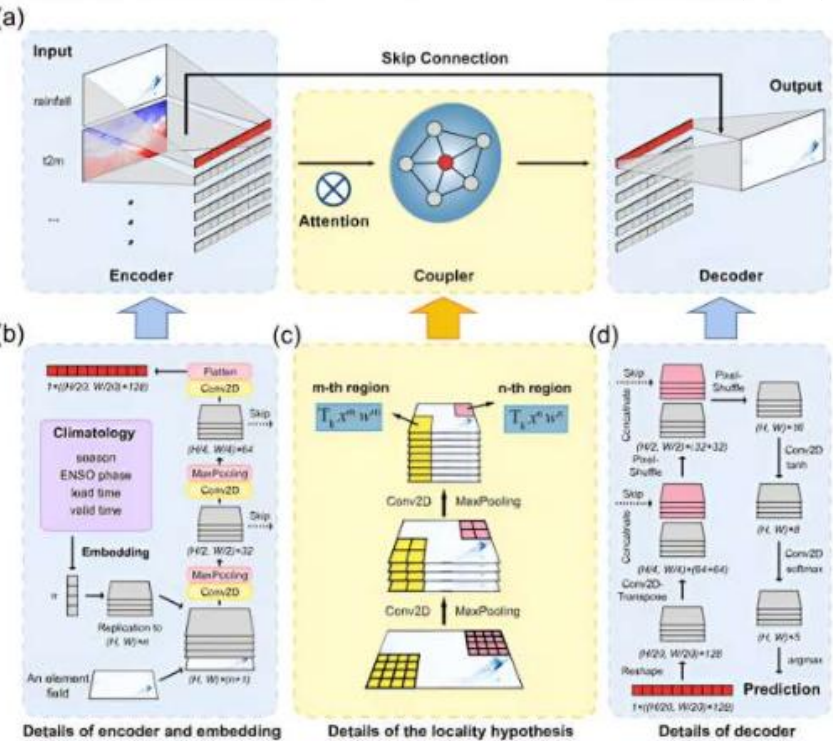
(slide inspired by Alexander Gaunt's talk on GNNs)

Graph Neural Networks are *everywhere*!

JANUARY 22, 2024

Novel graph neural network models enhance precipitation forecasting

by Chinese Academy of Sciences Geo. Res. Lett (2024)



VB The Machine GamesBeat Jobs Special Issue Become a Member | S

The Machine
Making sense of AI

DeepMind claims its AI improved Google Maps travel time estimates by up to 50%

Kyle Wiggers @Kyle_L_Wiggers September 3, 2020 7:00 AM

f t

BBC Sign in News Sport Reel Worklife Travel Future

NEWS

Home Video World UK Business Tech Science Stories Entertainment & Arts

BBC WORKLIFE Our new guide for getting ahead

Scientists discover powerful antibiotic using AI

21 February 2020

Share

Food Discovery with Uber Eats: Using Graph Learning to Power Recommendations

Ankit Jain, Isaac Liu, Ankur Sarda, and Piero Molino December 4, 2019



Today's Talk

Journal of the Royal Statistical Society

Series A: Statistics in Society



JOURNAL ARTICLE

E-STGCN: extreme spatio-temporal graph convolutional networks for air quality forecasting

Madhurima Panja, Tanujit Chakraborty , Anubhab Biswas, Soudeep Deb

[Author Notes](#)

Journal of the Royal Statistical Society Series A: Statistics in Society, qnag010,
<https://doi.org/10.1093/jrsssa/qnag010>

Published: 12 February 2026 **Article history** ▼



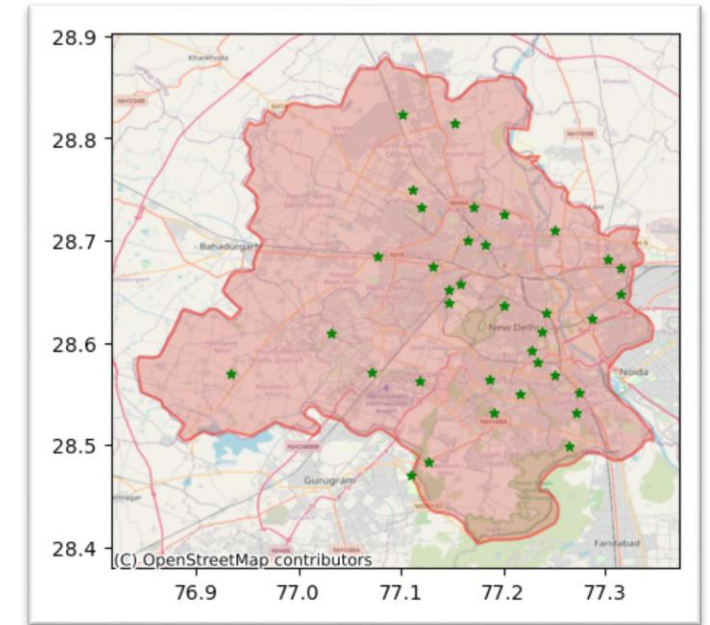
Air Quality: A Global Concern



- Emission of particulate matter and harmful gases in the air from industrialization degrades air quality levels leading to harmful health hazards.
- WHO has set National Ambient Air Quality Standards (NAAQS) for major air pollutants to protect the public health and environment.
- Despite several efforts, Delhi (Capital of India) remains the most polluted capital city in the world as of early 2024.

Air Quality Data of Delhi

- In efforts to monitor and reduce the air pollution levels, the Central Pollution Control Board (CPCB) in Delhi has installed a network of forty ground-based sensors at different strategic locations.
- The sensors collect near real-time (hourly) data on the concentration levels of major air pollutants, e.g., PM_{10} , $\text{PM}_{2.5}$, NO_2 .
- This timely information empowers the pollution control management system, facilitating informed decision-making to reduce pollution levels.



Delhi Air Quality Data Characteristics

Daily pollutant concentration levels of **PM_{2.5}**, **PM₁₀**, **NO₂** were obtained from 40 ground-based sensors of Delhi's CPCB during 1st January 2019 till 31st December 2023.

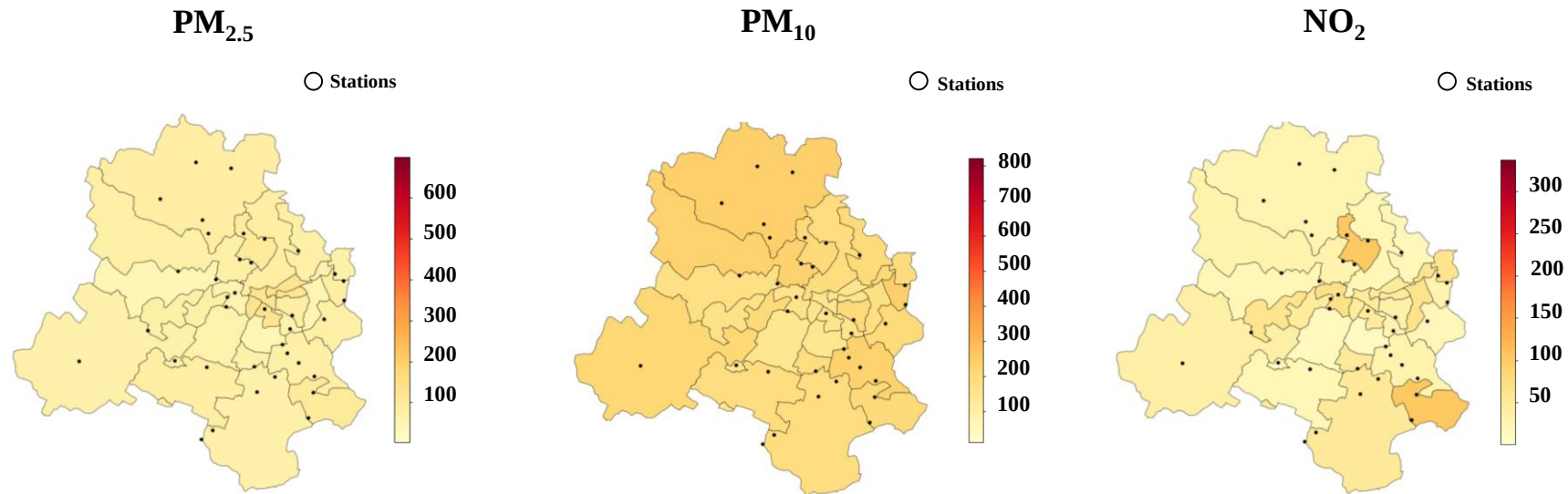


Fig: Evolution of Pollution Concentration recorded in Delhi during 2019 – 2022*.

- ✓ Nonlinear
- ✓ Non-stationary
- ✓ Long-term dependency
- ✓ Seasonality
- ✓ Extreme events

*NAAQS Thresholds for PM_{2.5} is 60 $\mu\text{g}/\text{m}^3$, PM₁₀ is 100 $\mu\text{g}/\text{m}^3$, and NO₂ is 80 $\mu\text{g}/\text{m}^3$ ($\mu\text{g}/\text{m}^3$ - micrograms per cubic meter air).

Extreme Value Theory

- In classical statistics: focus on average behavior of stochastic process (Central Limit Theorem)
- In extreme value theory: focus on extreme and rare events (Fisher-Tippett Theorem)
- Block Maxima Method

$$M_n = \max\{X_1, \dots, X_n\}$$

M_n follows a Generalized Extreme Value distribution

- Peak over Threshold (PoT)

$$\{X_i - \tau \mid X_i > \tau\}$$

very large threshold τ follow a Generalized Pareto distribution (GPD)

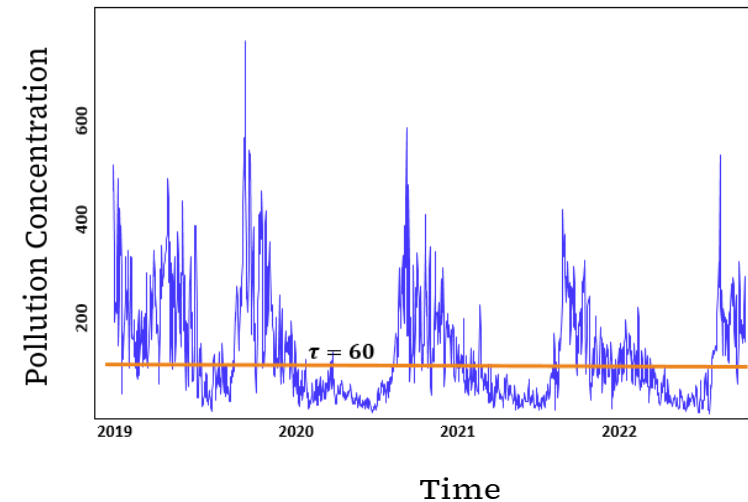


Fig: PoT method on PM_{2.5} level in Alipur, Delhi

Extreme Values in AQI Data

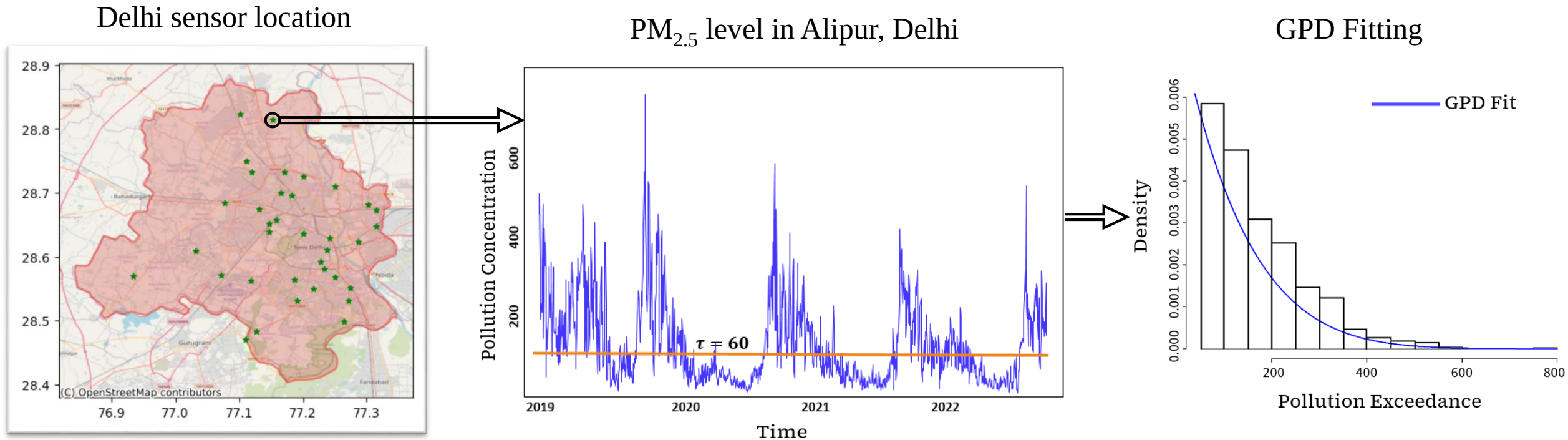


Fig: PoT analysis of PM_{2.5} concentration level recorded daily during 2019 – 2022 at Alipur, Delhi

- In Delhi, the air pollution level increases with the onset of winter due to the land-locked geographical location and low temperature restricting the dispersion of the pollutants (Extreme measurements arise).
- How can we accurately forecast AQIs for Delhi in the presence of extremes?

Proposed Solution: E-STGCN

- We utilize PoT approach from extreme value theory to provide prior information to the STGCN about exceedances of air pollutant concentration levels over the NAAQS thresholds.
- In a nutshell, we design the loss function of the temporal module of STGCN by incorporating a PoT-based loss with the conventional data loss when the in-sample predictions exceed NAAQS thresholds.
- **E-STGCN** better optimizes the model parameters to produce accurate forecasts in the presence of multiple peaks (**extreme values**) in the dataset.

Graph Neural Networks (GNNs)

- GNNs are an optimizable transformation on all attributes of the graph that preserves graph symmetries.

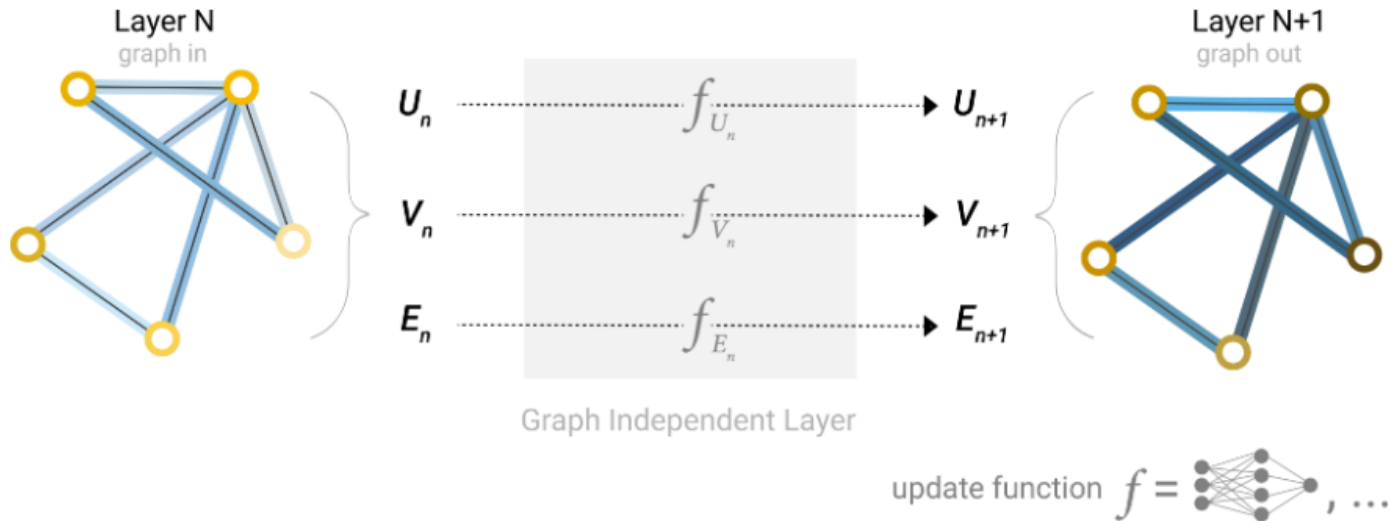


Fig: A single layer of a simple GNN.

- GNNs adopt a **graph-in, graph-out** architecture i.e., it accept a graph as input, with information loaded into its nodes (V), edges (E), global-context (U), and progressively transform these embeddings using distinct MLPs, without changing the connectivity of the input graph.

Neural message passing in GNNs

- GNNs utilize **neural message passing network** to exchange information amongst neighboring nodes and influence each other's updated embeddings as follows:
 - For each node in the graph, all the neighboring node embeddings (**messages**) are gathered.
 - All messages are aggregated via an **aggregate** function (like sum).
 - All pooled messages are passed through an **update function**, usually a learned neural network.

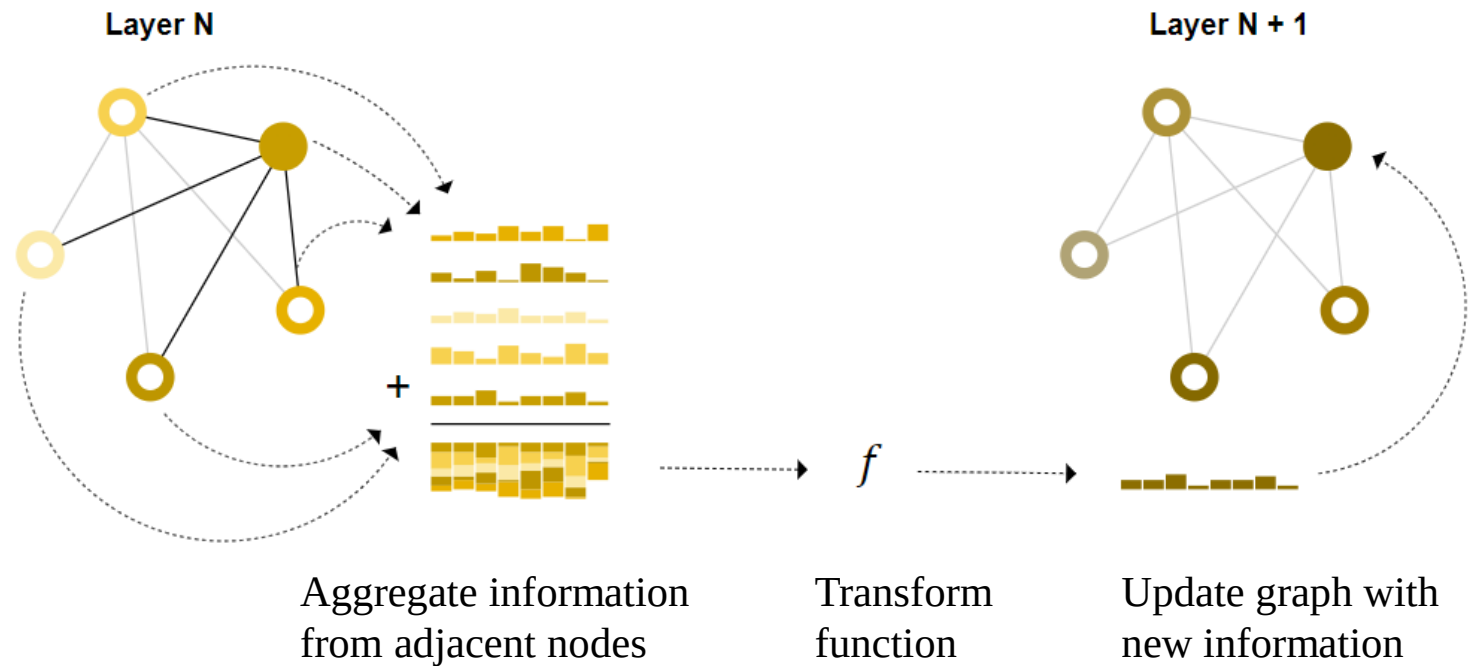


Fig: Updating the highlighted node embedding using adjacent nodes through neural message passing network.

Prediction task using GNNs

- The transformed graph embedded contains node information through GNN.
- For the prediction task we apply a regressor or classifier on each of the learned node representations.

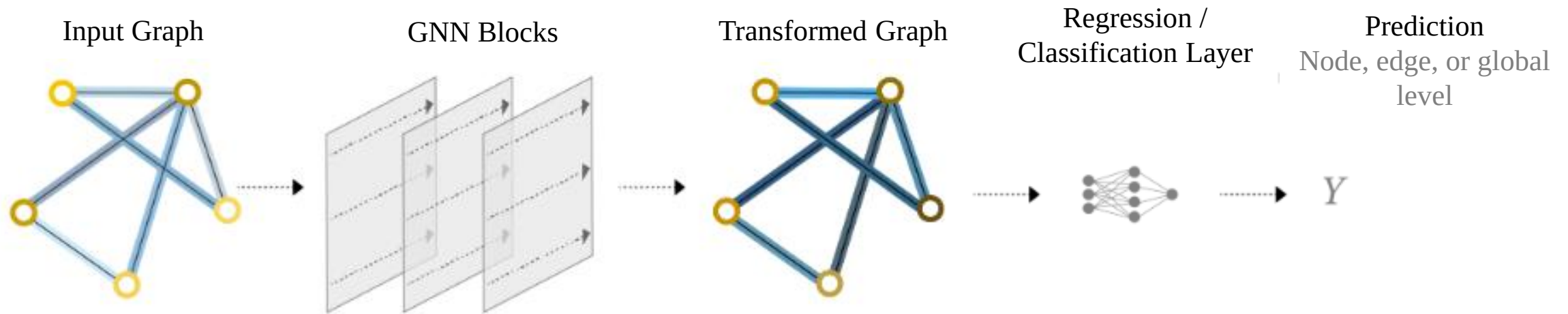


Fig: An end-to-end prediction-task using GNN.

Graph Convolutional Networks (GCNs)

GCNs architectures apply convolutional operations for information propagation between nodes in a graph and leverage localized aggregation of neighboring nodes' features. STGCN was proposed in 2018*.

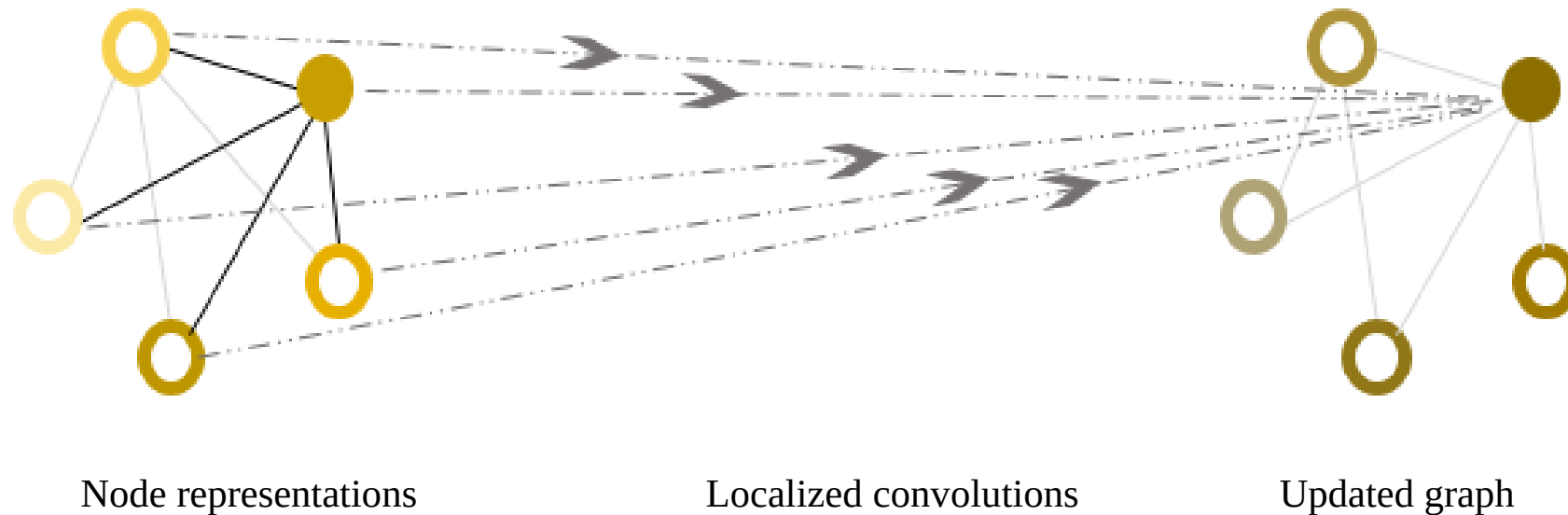


Fig: Updating the highlighted node embedding using localized convolution operation

*Yu, B., Yin, H., Zhu, Z. (2018). Spatio-temporal Graph Convolutional Networks: A Deep Learning Framework for Traffic Forecasting. *IJCAI*.

E-STGCN Framework

- **Input:** Pollutant concentrations across N sensors at timestamp t , $\mathbf{X}_t = \{X_t^1, \dots, X_t^N\} \in \mathbb{R}^N$; $t = 1, \dots, T$.
- **Output:** q (≥ 1) steps ahead forecast $\hat{\mathbf{X}}_{T+1}, \dots, \hat{\mathbf{X}}_{T+q}$ based on T historical observations.
- **Procedure:**
 - Design a spatiotemporal graph with nodes (sensor locations) and edges (interconnections).
 - The input can be represented as $\mathcal{G} = \{\mathcal{G}_1, \mathcal{G}_2, \dots, \mathcal{G}_T\}$ with $\mathcal{G}_t = \{\mathbf{X}_t, A\}$ being the graph at time t and $A \in \mathbb{R}^{N \times N}$ being the static adjacency matrix.
 - Compute $A = [a_{ij}]_{N \times N}$ using the **weighted Haversine distance** as

$$d_{ij} = 2R \sin^{-1} \left[\sqrt{\sin^2 \left(\frac{\phi^i - \phi^j}{2} \right) + \cos(\phi^i) \cos(\phi^j) \sin^2 \left(\frac{\lambda^i - \lambda^j}{2} \right)} \right]$$
$$a_{ij} = \exp \left(-\frac{d_{ij}^2}{\sigma^2} \right), \text{ when } i \neq j, \exp \left(-\frac{d_{ij}^2}{\sigma^2} \right) \geq \epsilon$$

where ϕ^i, λ^i is the latitude, longitude of sensor i , with σ^2, ϵ controlling the sparsity of A , and R represents Earth's radius.

E-STGCN: Spatial Module

- Encode the structural and feature-based aspects of the graph using GCNs (K layers) through neural message passing mechanism for node i as follows:

$$h_t^{i,(0)} = X_t^i$$
$$h_t^{i,(k)} = f_t^{(k)} \left(W_t^{(k)} \frac{\sum_{j \in \mathcal{N}(i)} h_t^{j,(k-1)}}{|\mathcal{N}(i)|} + B_t^{(k)} h_t^{i,(k-1)} \right); k = 1, 2, \dots, K$$

- Generate the spatiotemporal embedding based on the GCNs representations using an MLP as:

$$Z_t^i = \text{Dense} \left(h_t^{i,(K)} \right)$$

E-STGCN: Temporal Module

- Provide the spatiotemporal embedding $\mathbf{Z}_t = \{Z_t^1, \dots, Z_t^N\}$ to the [Temporal Block: LSTM layers](#) and a [Dense layer](#) to generate the desired forecasts.
- The mathematical formulation of the chain-like architecture in LSTM with input gate ($\mathbf{I}_t$), forget gate ($\mathbf{F}_t$), and output gate ($\mathbf{O}_t$) can be expressed as:

$$\begin{aligned}\mathbf{F}_t &= \text{ReLU}(U^F \mathbf{Y}_t + B^F); \mathbf{I}_t = \text{ReLU}(U^I \mathbf{Y}_t + B^I); \\ \mathbf{M}_t &= \tanh(U^M \mathbf{Y}_t + B^M); \mathbf{C}_t = (\mathbf{F}_t \odot \mathbf{C}_{t-1} \oplus \mathbf{I}_t \odot \mathbf{M}_t); \\ \mathbf{O}_t &= \text{ReLU}(U^O \mathbf{Y}_t + B^O); \mathbf{H}_t = (\mathbf{O}_t \odot \tanh(\mathbf{C}_t))\end{aligned}$$

where $\mathbf{Y}_t$ is the concatenation matrix with p lagged values of $\mathbf{Z}_t$ and previous hidden state $\mathbf{H}_{t-1}$, the new memory $\mathbf{M}_t$ at time t is used to update the cell state $\mathbf{C}_t$ (long term memory) and hidden state $\mathbf{H}_t$ (short term memory).

- Based on the computed values of $\mathbf{C}_t$ and $\mathbf{H}_t$ the q steps ahead forecast of $\mathbf{Z}_t$ is computed as:

$$\{\hat{\mathbf{Z}}_{t+1}, \hat{\mathbf{Z}}_{t+2}, \dots, \hat{\mathbf{Z}}_{t+q}\} = \text{ReLU}(U^H \mathbf{H}_t + B^H)$$

with U and B represents the recurrent weights matrix and the bias vector to be optimized based on the [proposed loss function](#).

E-STGCN: EVT Module

- EVT module performs PoT approach on the pollutant data.
- Hence, the conditional GPD for the i^{th} monitoring station at time t is:

$$P[X_t^i - \tau \leq x_t^i \mid X_t^i \geq \tau] = \begin{cases} 1 - \left(1 + \frac{\xi^i x_t^i}{\alpha^i}\right)^{-1/\xi_t^i}, & \xi^i \neq 0 \\ 1 - \exp\left(-\frac{x_t^i}{\alpha^i}\right)^2, & \xi^i = 0 \end{cases}$$

where $\xi^i (\in \mathbb{R})$ is the shape parameter and $\alpha^i (> 0)$ is the scale parameter.

- The POTL loss function is the deviance or **negative log-likelihood** loss from GPD

$$PoTL(z_t^i) = \text{NLL}(\text{GPD}) = \log g(\hat{\alpha}^i) + \left(1 + \frac{1}{\hat{\xi}^i}\right) \log g\left(1 + \frac{\hat{\xi}^i z_t^i}{\hat{\alpha}^i}\right)$$

for location i for one data point. **Lower NLL = Better Model Fit**

E-STGCN: Optimization

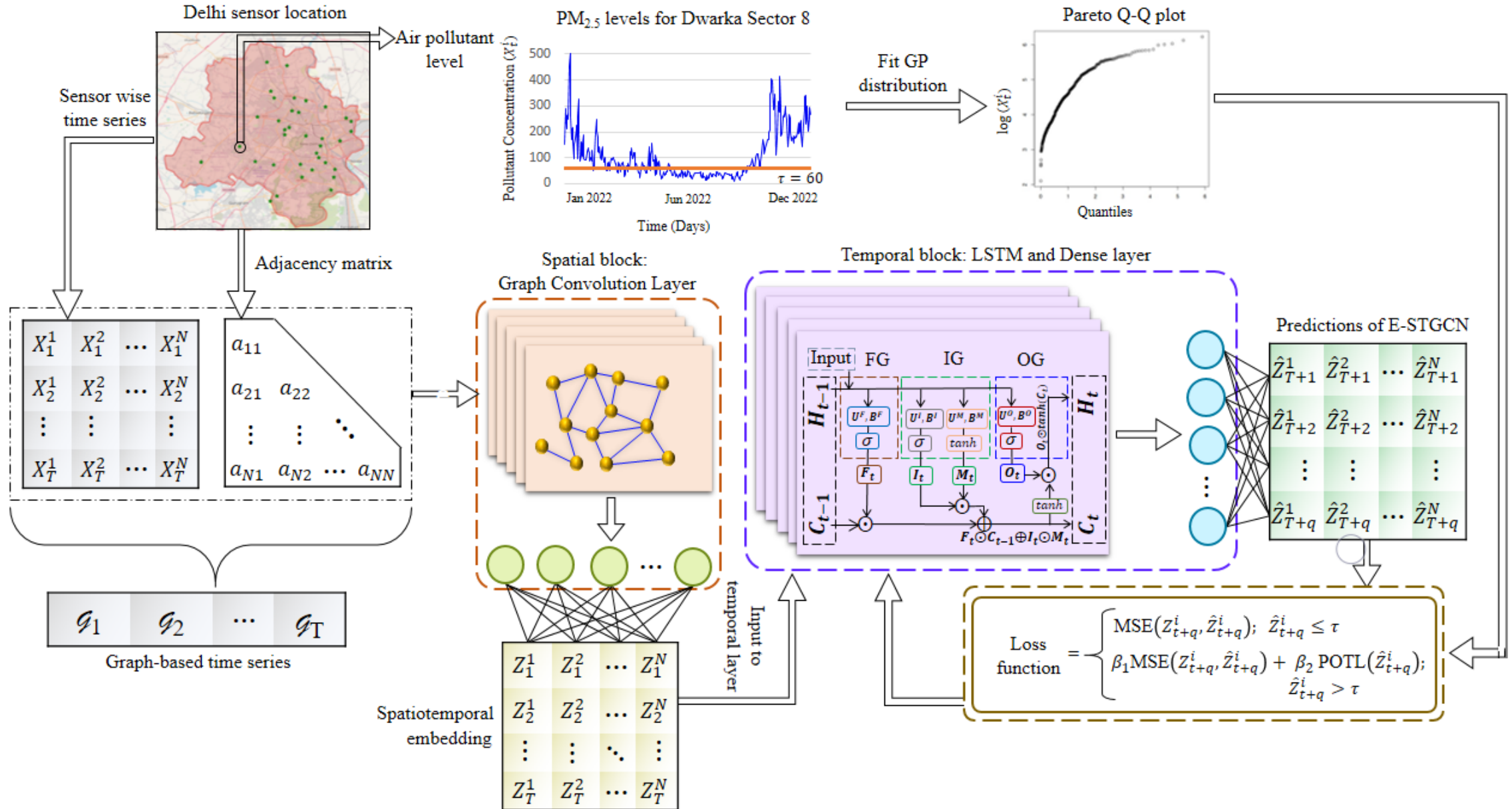
- Optimize the network using the following loss function (based on the [data loss](#) and the [PoT-based loss](#)):

$$\text{Loss}(Z_{t+q}^i, \hat{Z}_{t+q}^i) = \begin{cases} \text{MSE}(Z_{t+q}^i, \hat{Z}_{t+q}^i), & \hat{Z}_{t+q}^i \leq \tau, \\ \beta_1 \text{MSE}(Z_{t+q}^i, \hat{Z}_{t+q}^i) + \beta_2 \text{POTL}(\hat{Z}_{t+q}^i), & \hat{Z}_{t+q}^i > \tau, \end{cases}$$

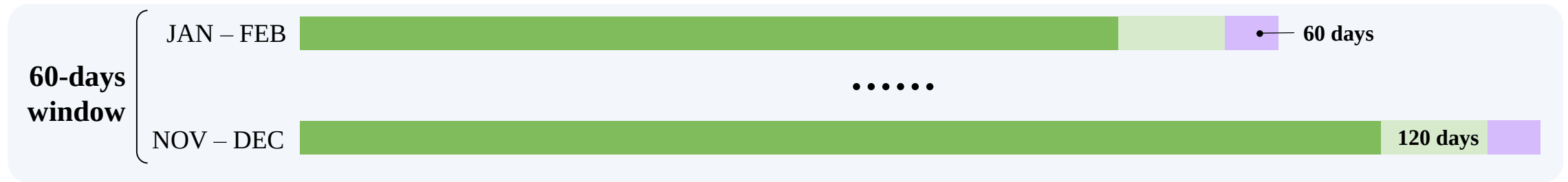
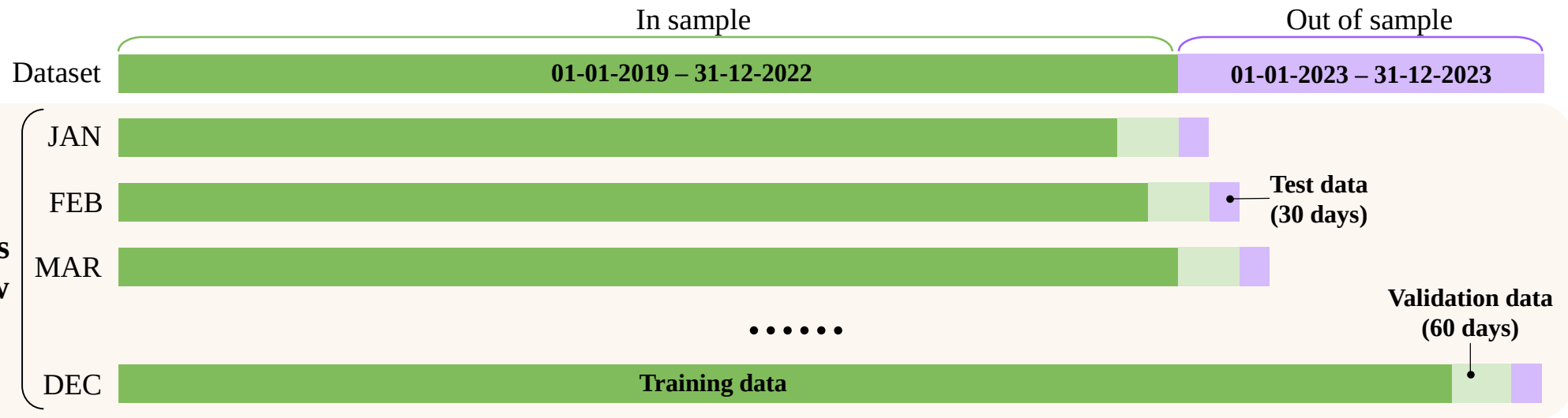
Hyperparameters
(cross validation)

- Following PINN, the final loss function of the LSTM network is formulated as a combination of the data loss and the [POT-based loss function whenever the predicted values exceed the NAAQS threshold \$\tau\$](#) .
- Since the loss function is differentiable almost everywhere, we employ the backpropagation approach to train the corresponding weights.
- The modified loss function employed for handling the temporal dependency in the E-STGCN framework integrates prior distributional knowledge to the predictions in case of threshold exceedances.

E-STGCN Architecture



Experimental Setup



Baseline Models

Models	Spatio-Temporal	Nonlinear	Nonstationary	Probabilistic Forecasting	Scalability	EV Handling
ARIMA	✗	✗	✓	✓	✗	✗
LSTM	✗	✓	✓	✗	✓	✗
TCN	✗	✓	✓	✗	✓	✗
DeepAR	✗	✓	✓	✗	✓	✗
Transformers	✗	✓	✓	✗	✓	✗
NBEATS	✗	✓	✓	✗	✓	✗
STARMA	✓	✗	✗	✓	✗	✗
GSTAR	✓	✗	✓	✗	✗	✗
GpGp	✓	✗	✗	✓	✓	✗
STNN	✓	✓	✓	✗	✗	✗
STGCN	✓	✓	✓	✗	✓	✗
DeepKrigging	✓	✓	✓	✓	✗	✗
E-STGCN	✓	✓	✓	✓	✓	✓

Statistical Significance

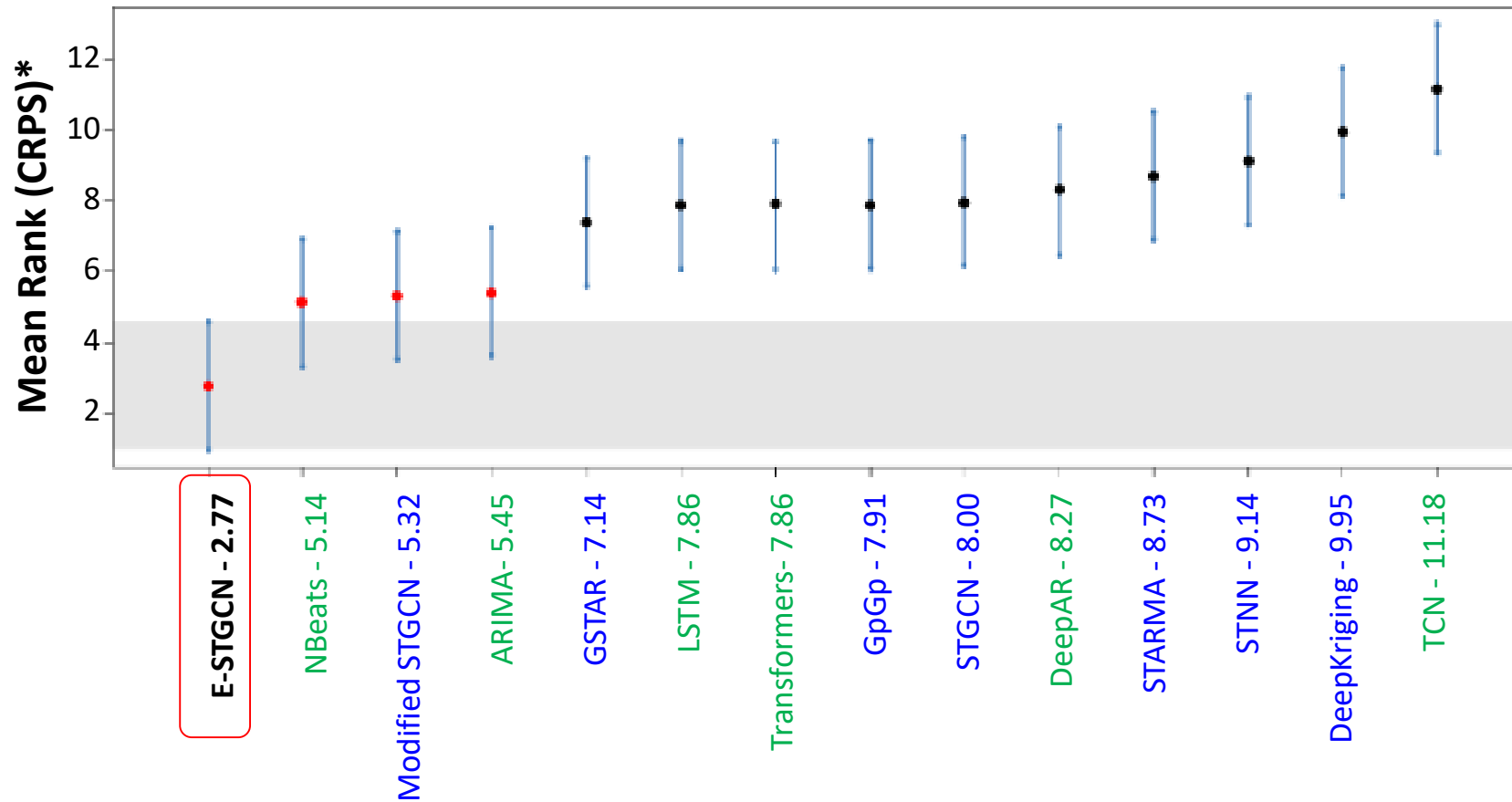


Fig: MCB test comparing the performance of E-STGCN with **spatiotemporal** and **temporal** models for PM_{2.5} forecasting based on CRPS metric

*Continuous Ranked Probability Score (CRPS): Gneiting, T., & Raftery, A. E. (2007). Strictly proper scoring rules, prediction, and estimation. *Journal of the American Statistical Association*, 102(477), 359-378.

Uncertainty Quantification

- Going **beyond point forecast** by providing valid prediction intervals for individual forecasts.
- E-STGCN leverages the distribution-free **Conformal Prediction*** (CP) technique for estimating uncertainty in predictions.
- In CP method, **conformal score** (S_t) is calculated based on the absolute residuals standardized with the conditional absolute deviation (uncertainty model: $\hat{Y}_t^U$)

$$S_t = \frac{|Y_t - \hat{Y}_t|}{\hat{Y}_t^U}$$

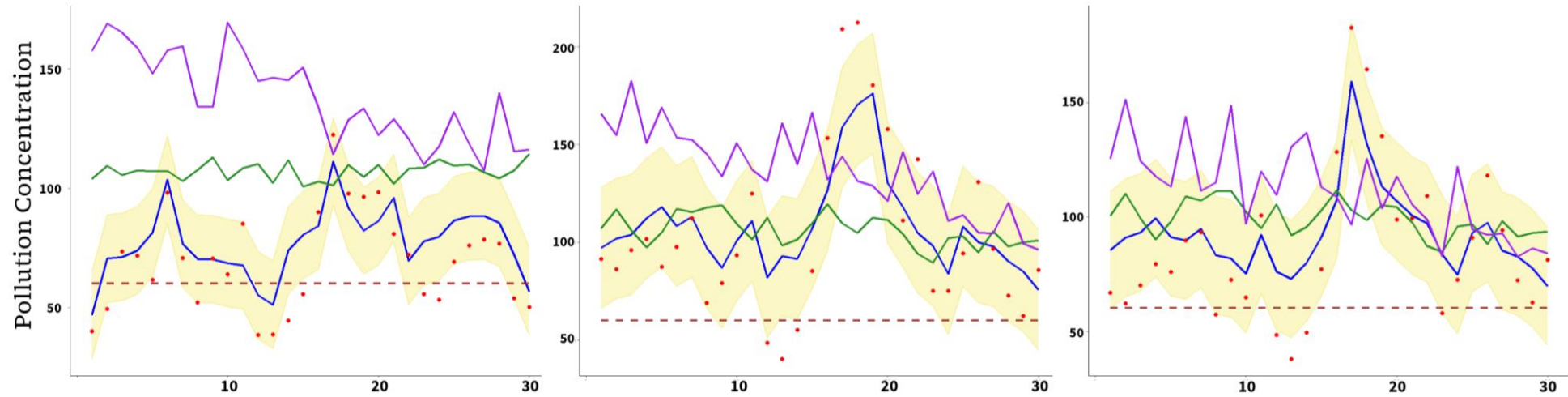
- The CP interval for prediction $\hat{Y}_t$ at time step t can be computed using the **conformal quantiles** ($\hat{\mathbb{Q}}_t$) as:

$$\mathbb{C}(\hat{Y}_t) = [\hat{Y}_t - \hat{\mathbb{Q}}_t \hat{Y}_t^U, \hat{Y}_t + \hat{\mathbb{Q}}_t \hat{Y}_t^U]$$

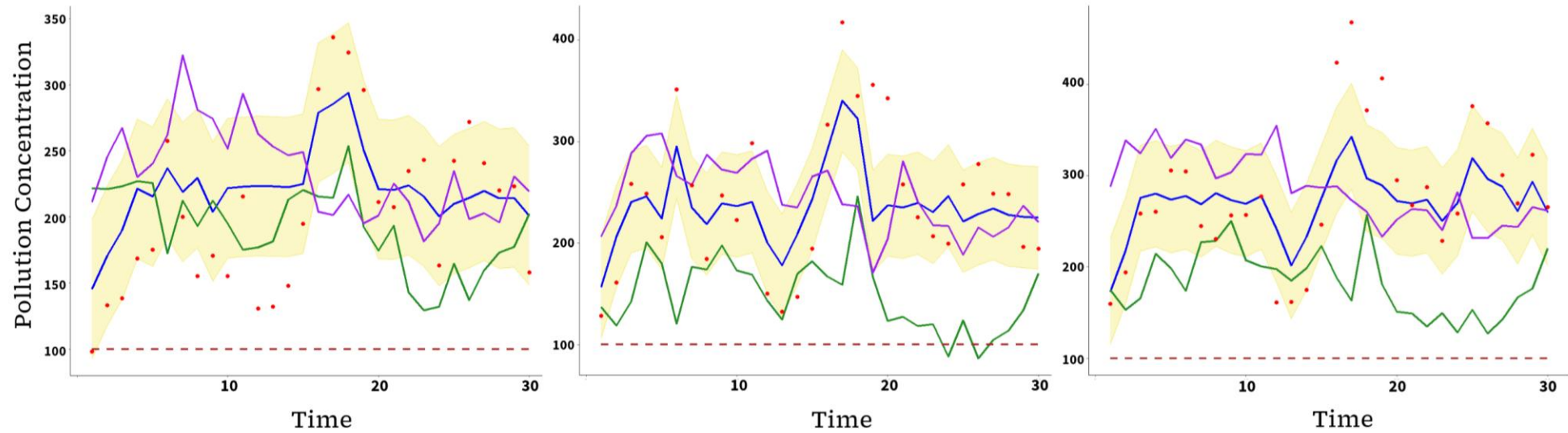
*Xu, Chen, and Yao Xie. "Conformal prediction interval for dynamic time-series." *ICML* 2021.

E-STGCN Prediction (Feb 2023)

$\text{PM}_{2.5}$



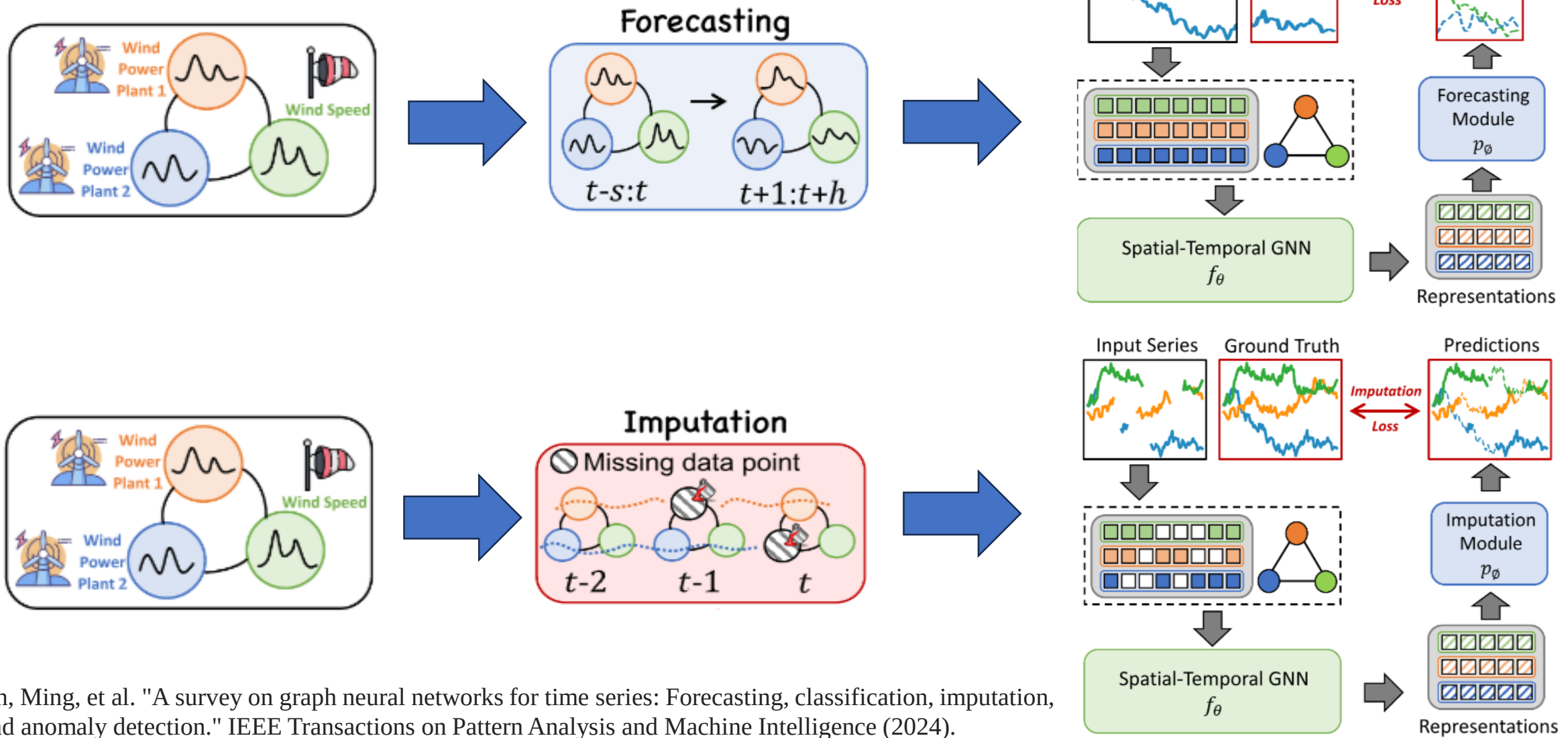
PM_{10}



Discussion

- The highlight of this work is the combination of deep-learning architecture and extreme value theory. Instead of the MSE loss, a modified loss function, which aggregate the MSE loss and the Peaks Over Threshold (POT) loss, is introduced as the objective function of E-STGCN.
- Empirical results using real-world air pollution data from 40 sensors in Delhi showcase the superiority of E-STGCN compared to baseline forecasters when extreme value theory is incorporated.
- The inclusion of probabilistic bands associated with E-STGCN point forecasts offers policymakers valuable insights about forecast uncertainty.
- Future Work (Spatio-temporal modeling of epidemics): A potential future research direction is extending E-STGCN to incorporate climatic interactions and using pollution transmission data to understand the spread of air-borne diseases.

Other Applications of GNNs for spatiotemporal data

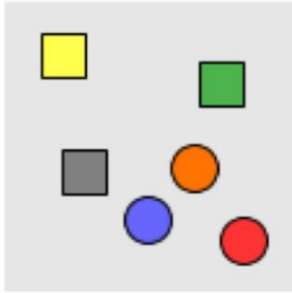


Jin, Ming, et al. "A survey on graph neural networks for time series: Forecasting, classification, imputation, and anomaly detection." IEEE Transactions on Pattern Analysis and Machine Intelligence (2024).

Open Problems

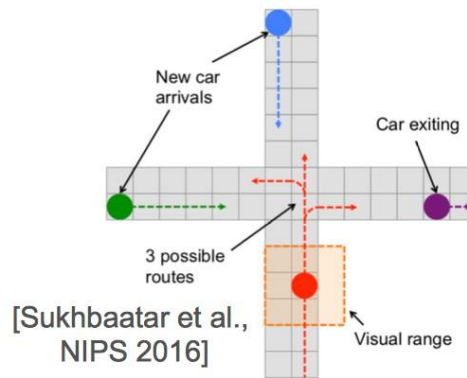
- Deep learning on graphs works and is very effective!
- Exciting area: lots of new applications and extensions (hard to keep up)

Relational reasoning



[Santoro et al., NIPS 2017]

Multi-Agent RL



[Sukhbaatar et al.,
NIPS 2016]

GCN for recommendation on 16 billion edge graph!



Source pin

[Leskovec lab, Stanford]



SUCCESSFUL
RECOMMENDATION



BAD RECOMMENDATION

Open problems:

- Theory
- Scalable, stable generative models ([Pathak and Chakraborty, 2026](#))
- Learning on large, evolving data
- Multi-modal and cross-model learning (e.g., sequence2graph)

**T
H
A
N
K

Y
O
U
!**



essentially,
all models are wrong,
but some are useful

George E. P. Box

Scan me
for Slides:

